

Adversarial Search and Recursion

The Minimax Algorithm and Solving Games

Martin Benning

University College London

ENGF0034 – Design and Professional Skills

Lecture Overview

Objectives

To explore the importance of understanding fundamental algorithms. To master the concept of recursion and apply it to the domain of Adversarial Search (Game AI). We will analyze the Minimax algorithm in detail and dissect a complete Python implementation for creating an unbeatable Tic-Tac-Toe AI, incorporating principles of Test-Driven Development.

- 1 Motivation: Understanding Fundamentals
- 2 The Principle of Recursion
- 3 Adversarial Search and Game Trees
- 4 The Minimax Algorithm
- 5 Implementation Analysis: The Board (oxo.py)
- 6 Implementation Analysis: The Minimax AI (oxo.py)
- 7 Testing and Analysis
- 8 Summary

Motivation: Understanding Fundamentals

Motivation: Understanding Fundamentals

The Philosophy of "The Lift" (Alan Kay)

Alan Kay, a Turing Award winner and pioneer of OOP and GUIs, often uses the metaphor of an elevator ("The Lift") to distinguish between users and engineers.

The User Perspective

- Uses the elevator functionally: presses a button, arrives at a destination.
- Relies on abstraction. Does not need to understand the underlying mechanics (motors, safety systems, algorithms).

The Engineer Perspective

- Understands the underlying principles.
- Capable of design, innovation, and solving fundamental problems.

The Philosophy of "The Lift" (Alan Kay)

Alan Kay, a Turing Award winner and pioneer of OOP and GUIs, often uses the metaphor of an elevator ("The Lift") to distinguish between users and engineers.

The User Perspective

- Uses the elevator functionally: presses a button, arrives at a destination.
- Relies on abstraction. Does not need to understand the underlying mechanics (motors, safety systems, algorithms).

The Engineer Perspective

- Understands the underlying principles.
- Capable of design, innovation, and solving fundamental problems.

The Philosophy of "The Lift" (Alan Kay)

Alan Kay, a Turing Award winner and pioneer of OOP and GUIs, often uses the metaphor of an elevator ("The Lift") to distinguish between users and engineers.

The User Perspective

- Uses the elevator functionally: presses a button, arrives at a destination.
- Relies on abstraction. Does not need to understand the underlying mechanics (motors, safety systems, algorithms).

The Engineer Perspective

- Understands the underlying principles.
- Capable of design, innovation, and solving fundamental problems.

Today's Topic: Recursion and Adversarial Search

Going Inside the "AI Lift"

Today we look beyond high-level AI libraries and delve into the fundamental algorithms that power decision-making in competitive environments.

Key Concepts We Will Explore

- **Recursion:** The mechanism and its application.
- **Adversarial Search:** Modeling competitive environments.
- **The Minimax Algorithm:** The foundation of optimal decision making in games.
- **Implementation and Testing (TDD):** Analyzing a robust Python solution.

Today's Topic: Recursion and Adversarial Search

Going Inside the "AI Lift"

Today we look beyond high-level AI libraries and delve into the fundamental algorithms that power decision-making in competitive environments.

Key Concepts We Will Explore

- **Recursion:** The mechanism and its application.
- **Adversarial Search:** Modeling competitive environments.
- **The Minimax Algorithm:** The foundation of optimal decision making in games.
- **Implementation and Testing (TDD):** Analyzing a robust Python solution.

The Principle of Recursion

The Principle of Recursion

What is Recursion?

Definition

Recursion is a method of solving a computational problem where the solution depends on solutions to smaller instances of the same problem. In programming, this means a function that calls itself.

The Structure of a Recursive Function

Every correct recursive implementation requires two essential components:

1. **Base Case:** The simplest instance of the problem, solved directly. This stops the recursion.
2. **Recursive Step:** The function calls itself with a modified input that moves closer to the base case.

The Danger

What is Recursion?

Definition

Recursion is a method of solving a computational problem where the solution depends on solutions to smaller instances of the same problem. In programming, this means a function that calls itself.

The Structure of a Recursive Function

Every correct recursive implementation requires two essential components:

1. **Base Case:** The simplest instance of the problem, solved directly. This stops the recursion.
2. **Recursive Step:** The function calls itself with a modified input that moves closer to the base case.

The Danger

What is Recursion?

Definition

Recursion is a method of solving a computational problem where the solution depends on solutions to smaller instances of the same problem. In programming, this means a function that calls itself.

The Structure of a Recursive Function

Every correct recursive implementation requires two essential components:

1. **Base Case:** The simplest instance of the problem, solved directly. This stops the recursion.
2. **Recursive Step:** The function calls itself with a modified input that moves closer to the base case.

The Danger

The Mechanism: The Call Stack

Recursion relies heavily on the program's call stack to manage execution flow and local state.

How it Works

- 1 When a function is called (including recursively), a new **Stack Frame** is pushed onto the call stack.
- 2 This frame stores local variables, parameters, and the return address.
- 3 The recursion "winds" as frames are pushed until the base case is reached.
- 4 The recursion "unwinds" as the base case returns, and frames are popped off the stack sequentially, returning values to the caller.

Implications

Recursion consumes stack memory proportional to the maximum depth ($\mathcal{O}(D)$)

The Mechanism: The Call Stack

Recursion relies heavily on the program's call stack to manage execution flow and local state.

How it Works

- 1 When a function is called (including recursively), a new **Stack Frame** is pushed onto the call stack.
- 2 This frame stores local variables, parameters, and the return address.
- 3 The recursion "winds" as frames are pushed until the base case is reached.
- 4 The recursion "unwinds" as the base case returns, and frames are popped off the stack sequentially, returning values to the caller.

Implications

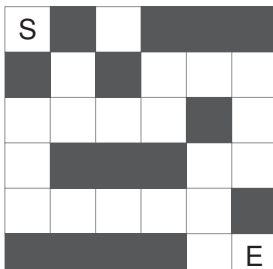
Recursion consumes stack memory proportional to the maximum depth ($\mathcal{O}(D)$)

A Concrete Application: Solving a Maze

Recursion is highly effective for problems involving exploration and search, such as solving a maze. This is a single-agent search problem.

The Problem

Given a maze (represented as a grid or graph), find a path from a Start location (S) to an End location (E).

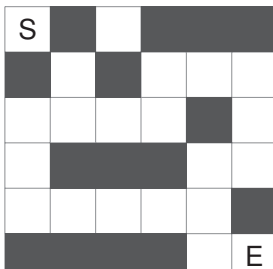


A Concrete Application: Solving a Maze

Recursion is highly effective for problems involving exploration and search, such as solving a maze. This is a single-agent search problem.

The Problem

Given a maze (represented as a grid or graph), find a path from a Start location (S) to an End location (E).



The Naive Recursive Maze Algorithm

We can formalize this using a recursive function 'Solve(Location)'. This approach naturally implements a **Depth-First Search (DFS)**.

Algorithm Sketch (Naive)

Base Case 1 (Success): If 'Location == End', return 'True'.

Base Case 2 (Failure): If 'Location' is a Wall/Boundary, return 'False'.

Recursive Step: For each neighbor (e.g., N, E, S, W):

- Recursively call 'Solve(Neighbor)'.
- If the call returns 'True', then this neighbor is on the solution path. Return 'True'.

Exhaustion (Dead End): If all neighbors have been tried and none lead to a solution, return 'False'.

The Critical Flaw

The Naive Recursive Maze Algorithm

We can formalize this using a recursive function 'Solve(Location)'. This approach naturally implements a **Depth-First Search (DFS)**.

Algorithm Sketch (Naive)

Base Case 1 (Success): If 'Location == End', return 'True'.

Base Case 2 (Failure): If 'Location' is a Wall/Boundary, return 'False'.

Recursive Step: For each neighbor (e.g., N, E, S, W):

- Recursively call 'Solve(Neighbor)'.
- If the call returns 'True', then this neighbor is on the solution path. Return 'True'.

Exhaustion (Dead End): If all neighbors have been tried and none lead to a solution, return 'False'.

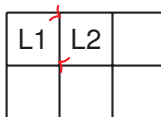
The Critical Flaw

The Challenge of Cycles and Infinite Recursion

The Problem: Lack of Memory

The naive algorithm does not track where it has been during the current exploration path.

- 1 We move from L1 to L2.
- 2 We recursively call 'Solve(L2)'.
- 3 L2 checks its neighbors, which includes L1 (since it is not a wall).
- 4 We move from L2 back to L1.
- 5 We recursively call 'Solve(L1)'...

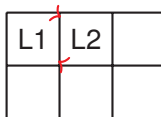


The Challenge of Cycles and Infinite Recursion

The Problem: Lack of Memory

The naive algorithm does not track where it has been during the current exploration path.

- 1 We move from L1 to L2.
- 2 We recursively call 'Solve(L2)'.
- 3 L2 checks its neighbors, which includes L1 (since it is not a wall).
- 4 We move from L2 back to L1.
- 5 We recursively call 'Solve(L1)'...



The Solution: Marking Visited Locations

To prevent cycles, we must remember the path taken so far during the current exploration. We need to maintain state.

The Strategy: Dropping Breadcrumbs

When entering a location, mark it as "Visited" (part of the current path). We must augment the state space being searched.

The Refined Algorithm (Solve(Location))

We introduce a new Base Case to ensure termination.

- 1 If Location is End, return 'True'.
- 2 If Location is Wall or **Visited**, return 'False'.
- 3 Mark Location as **Visited**.
- 4 For each Neighbor:

The Solution: Marking Visited Locations

To prevent cycles, we must remember the path taken so far during the current exploration. We need to maintain state.

The Strategy: Dropping Breadcrumbs

When entering a location, mark it as "Visited" (part of the current path). We must augment the state space being searched.

The Refined Algorithm (Solve(Location))

We introduce a new Base Case to ensure termination.

- 1 If Location is End, return 'True'.
- 2 If Location is Wall or **Visited**, return 'False'.
- 3 Mark Location as **Visited**.
- 4 For each Neighbor:

The Mechanism of Backtracking

What happens when the algorithm reaches a dead end and returns 'False' (Step 5)?

Backtracking

When a recursive call returns 'False', the execution returns to the previous stack frame (the previous location). The algorithm then attempts the next available direction from that previous location (the loop in Step 4 continues). This process of exploration and subsequent retraction, managed implicitly by the call stack, is known as **Backtracking**.

Refinement: Unmarking for Path Reconstruction

If we want to reconstruct the actual path (not just its existence), or allow a square to be part of an alternative path later, we must "unmark" the cell when backtracking.

- When returning 'False' (Step 5), change the mark from "Visited" back to "Open".
- Alternatively, for optimization, we can mark it as a "Dead End" if we only care about

The Mechanism of Backtracking

What happens when the algorithm reaches a dead end and returns 'False' (Step 5)?

Backtracking

When a recursive call returns 'False', the execution returns to the previous stack frame (the previous location). The algorithm then attempts the next available direction from that previous location (the loop in Step 4 continues). This process of exploration and subsequent retraction, managed implicitly by the call stack, is known as **Backtracking**.

Refinement: Unmarking for Path Reconstruction

If we want to reconstruct the actual path (not just its existence), or allow a square to be part of an alternative path later, we must "unmark" the cell when backtracking.

- When returning 'False' (Step 5), change the mark from "Visited" back to "Open".
- Alternatively, for optimization, we can mark it as a "Dead End" if we only care about

The Maze Algorithm: Pseudocode

Implementing the refined algorithm with backtracking (unmarking).

Listing 1: Recursive Maze Solver (Pseudocode)

```
1 def SolveMaze(x, y):  
2     # Base Case 1: Found the exit  
3     if IsExit(x, y): return True  
4  
5     # Base Case 2: Invalid move (Wall or already visited)  
6     if IsWall(x, y) or IsVisited(x, y): return False  
7  
8     # Mark current location as part of the path  
9     MarkVisited(x, y)  
10  
11     # Recursive Steps: Try all directions (N, E, S, W)  
12     if SolveMaze(x, y+1): return True # North  
13     if SolveMaze(x+1, y): return True # East
```

Maze Solving in Context (DFS vs BFS)

The recursive maze solver is a concrete implementation of Depth-First Search (DFS).

Characteristics of DFS (Recursive)

- **Deep Exploration:** Explores one path as far as possible before backtracking.
- **Frontier Management:** The recursion call stack implicitly manages the nodes to explore (LIFO Stack behaviour).
- **Space Complexity:** $\mathcal{O}(D)$ (stack depth), which is often efficient.
- **Optimality:** Does not guarantee finding the shortest path.

Comparison with Breadth-First Search (BFS)

- BFS explores layer by layer.
- **Frontier Management:** BFS uses a Queue (FIFO), typically implemented iteratively.

Maze Solving in Context (DFS vs BFS)

The recursive maze solver is a concrete implementation of Depth-First Search (DFS).

Characteristics of DFS (Recursive)

- **Deep Exploration:** Explores one path as far as possible before backtracking.
- **Frontier Management:** The recursion call stack implicitly manages the nodes to explore (LIFO Stack behaviour).
- **Space Complexity:** $\mathcal{O}(D)$ (stack depth), which is often efficient.
- **Optimality:** Does not guarantee finding the shortest path.

Comparison with Breadth-First Search (BFS)

- BFS explores layer by layer.
- **Frontier Management:** BFS uses a Queue (FIFO), typically implemented iteratively,

Adversarial Search and Game Trees

Adversarial Search and Game Trees

Introduction to Adversarial Search

In standard search (e.g., pathfinding), we seek an optimal path in a passive environment. In Adversarial Search (Games), we face rational opponents actively working against our goals.

The Challenge

We need a **strategy** that accounts for the opponent's moves, assuming the opponent is also playing optimally.

Focus: Two-Player Zero-Sum Games

We will study games with the following properties:

- **Two-Player** and **Zero-Sum** (One player's gain is the other's loss).
- **Deterministic** (No chance).
- **Perfect Information** (The full state is visible).

Introduction to Adversarial Search

In standard search (e.g., pathfinding), we seek an optimal path in a passive environment. In Adversarial Search (Games), we face rational opponents actively working against our goals.

The Challenge

We need a **strategy** that accounts for the opponent's moves, assuming the opponent is also playing optimally.

Focus: Two-Player Zero-Sum Games

We will study games with the following properties:

- **Two-Player** and **Zero-Sum** (One player's gain is the other's loss).
- **Deterministic** (No chance).
- **Perfect Information** (The full state is visible).

Introduction to Adversarial Search

In standard search (e.g., pathfinding), we seek an optimal path in a passive environment. In Adversarial Search (Games), we face rational opponents actively working against our goals.

The Challenge

We need a **strategy** that accounts for the opponent's moves, assuming the opponent is also playing optimally.

Focus: Two-Player Zero-Sum Games

We will study games with the following properties:

- **Two-Player** and **Zero-Sum** (One player's gain is the other's loss).
- **Deterministic** (No chance).
- **Perfect Information** (The full state is visible).

Modeling Games: The Game Tree

The fundamental data structure for analyzing these games is the Game Tree.

Definition

A Game Tree represents all possible sequences of moves from the starting state.

- **Nodes:** Represent game states (board configurations).
- **Edges:** Represent moves (actions).
- **Root:** The initial state.
- **Leaves (Terminal Nodes):** States where the game ends (Win/Loss/Draw).

Recursive Structure

The game tree is inherently recursive. The structure of the game starting from any node is itself a (smaller) game tree. This makes recursive algorithms ideal for traversal.

Modeling Games: The Game Tree

The fundamental data structure for analyzing these games is the Game Tree.

Definition

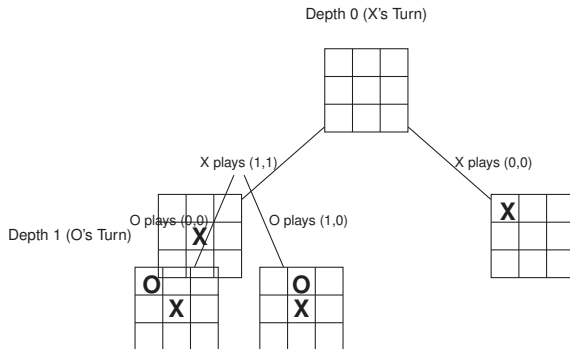
A Game Tree represents all possible sequences of moves from the starting state.

- **Nodes:** Represent game states (board configurations).
- **Edges:** Represent moves (actions).
- **Root:** The initial state.
- **Leaves (Terminal Nodes):** States where the game ends (Win/Loss/Draw).

Recursive Structure

The game tree is inherently recursive. The structure of the game starting from any node is itself a (smaller) game tree. This makes recursive algorithms ideal for traversal.

A Partial Tic-Tac-Toe Game Tree



Game Complexity and Feasibility

The feasibility of exploring the game tree depends on its size.

Metrics

- b : Branching factor (average number of legal moves).
- d : Depth (maximum length of the game).
- Game Tree Size (Time Complexity): $\mathcal{O}(b^d)$.

Tic-Tac-Toe Analysis

- $d = 9$. $b \leq 9$.
- Upper bound on Game Tree Size: $9! = 362,880$.
- This is small enough to search exhaustively. We can "solve" the game.

Game Complexity and Feasibility

The feasibility of exploring the game tree depends on its size.

Metrics

- b : Branching factor (average number of legal moves).
- d : Depth (maximum length of the game).
- Game Tree Size (Time Complexity): $\mathcal{O}(b^d)$.

Tic-Tac-Toe Analysis

- $d = 9$. $b \leq 9$.
- Upper bound on Game Tree Size: $9! = 362,880$.
- This is small enough to search exhaustively. We can "solve" the game.

Game Complexity and Feasibility

The feasibility of exploring the game tree depends on its size.

Metrics

- b : Branching factor (average number of legal moves).
- d : Depth (maximum length of the game).
- Game Tree Size (Time Complexity): $\mathcal{O}(b^d)$.

Tic-Tac-Toe Analysis

- $d = 9$. $b \leq 9$.
- Upper bound on Game Tree Size: $9! = 362,880$.
- This is small enough to search exhaustively. We can "solve" the game.

The Minimax Algorithm

The Minimax Algorithm

The Goal: Optimal Play Against a Rational Opponent

We want an algorithm that chooses the best possible move, assuming the opponent also plays optimally to defeat us.

Defining Utility and Players

MAX: The player trying to maximise the final utility score.

MIN: The opponent trying to minimise the final utility score.

We assign numerical values to terminal states from MAX's perspective (e.g., Win=+1, Draw=0, Loss=-1).

The Minimax Principle

"Maximise your minimum possible gain." MAX chooses the move that maximizes the final score, assuming MIN will subsequently play to minimize that score.

The Goal: Optimal Play Against a Rational Opponent

We want an algorithm that chooses the best possible move, assuming the opponent also plays optimally to defeat us.

Defining Utility and Players

MAX: The player trying to maximise the final utility score.

MIN: The opponent trying to minimise the final utility score.

We assign numerical values to terminal states from MAX's perspective (e.g., Win=+1, Draw=0, Loss=-1).

The Minimax Principle

"Maximise your minimum possible gain." MAX chooses the move that maximizes the final score, assuming MIN will subsequently play to minimize that score.

The Minimax Recurrence Relation

The Minimax value of a state s , $V(s)$, is defined recursively, mirroring the structure of the game tree.

Recursive Definition

$V(s) =$

- $Utility(s)$ if s is terminal (Base Case)
- $\max_{s' \in Successors(s)} V(s')$ if $Player(s) = MAX$ (Recursive Step)
- $\min_{s' \in Successors(s)} V(s')$ if $Player(s) = MIN$ (Recursive Step)

The Algorithm

Minimax uses Depth-First Search (DFS). Utilities are calculated at the leaves and propagated upwards according to this recurrence.

The Minimax Recurrence Relation

The Minimax value of a state s , $V(s)$, is defined recursively, mirroring the structure of the game tree.

Recursive Definition

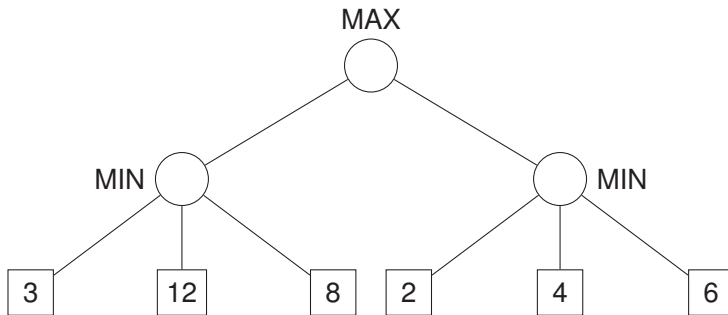
$V(s) =$

- $\text{Utility}(s)$ if s is terminal (Base Case)
- $\max_{s' \in \text{Successors}(s)} V(s')$ if $\text{Player}(s) = \text{MAX}$ (Recursive Step)
- $\min_{s' \in \text{Successors}(s)} V(s')$ if $\text{Player}(s) = \text{MIN}$ (Recursive Step)

The Algorithm

Minimax uses Depth-First Search (DFS). Utilities are calculated at the leaves and propagated upwards according to this recurrence.

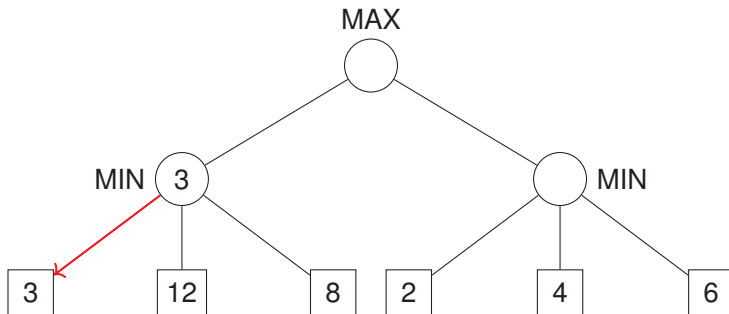
Visualising Minimax Propagation



Explanation

- Utilities are defined at the leaves.
- MIN selects the minimum utility from its children.

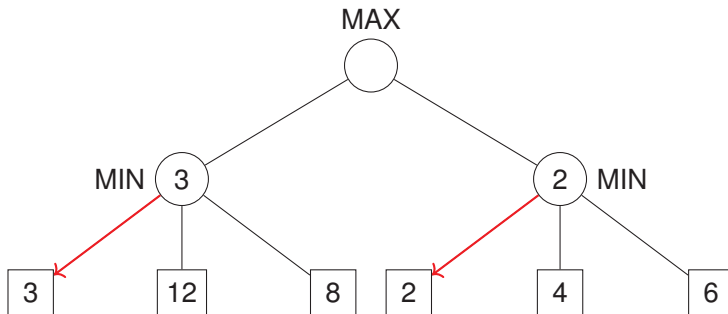
Visualising Minimax Propagation



Explanation

- Utilities are defined at the leaves.
- MIN selects the minimum utility from its children.

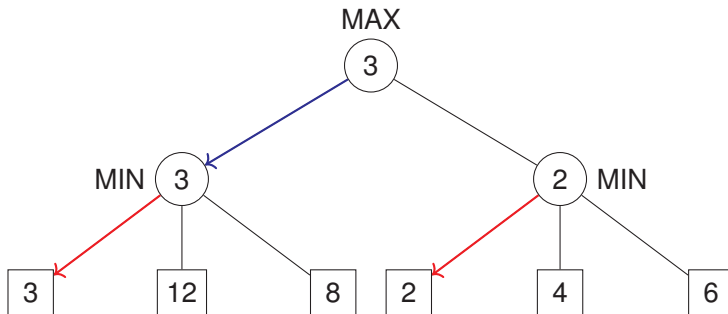
Visualising Minimax Propagation



Explanation

- Utilities are defined at the leaves.
- MIN selects the minimum utility from its children.

Visualising Minimax Propagation



Explanation

- Utilities are defined at the leaves.
- MIN selects the minimum utility from its children.

Implementation Analysis: The Board (oxo.py)

Implementation Analysis: The Board (oxo.py)

The Structure of the Implementation

The provided `oxo.py` separates the game mechanics (the rules) from the strategy (the AI). This is good design (Separation of Concerns).

The Board Class

Responsibilities: Maintain the state of the 3x3 grid and implement the rules (moves, win/draw checks).

The Oxo Class (The AI Engine)

Responsibilities: Implement the Minimax search algorithm and manage the game flow.

The Importance of Efficiency

The Board operations must be efficient as they are called repeatedly during the Minimax search ($\mathcal{O}(b^d)$ times).

The Structure of the Implementation

The provided `oxo.py` separates the game mechanics (the rules) from the strategy (the AI). This is good design (Separation of Concerns).

The Board Class

Responsibilities: Maintain the state of the 3x3 grid and implement the rules (moves, win/draw checks).

The Oxo Class (The AI Engine)

Responsibilities: Implement the Minimax search algorithm and manage the game flow.

The Importance of Efficiency

The Board operations must be efficient as they are called repeatedly during the Minimax search ($\mathcal{O}(b^d)$ times).

The Structure of the Implementation

The provided `oxo.py` separates the game mechanics (the rules) from the strategy (the AI). This is good design (Separation of Concerns).

The Board Class

Responsibilities: Maintain the state of the 3x3 grid and implement the rules (moves, win/draw checks).

The Oxo Class (The AI Engine)

Responsibilities: Implement the Minimax search algorithm and manage the game flow.

The Importance of Efficiency

The Board operations must be efficient as they are called repeatedly during the Minimax search ($\mathcal{O}(b^d)$ times).

Board Representation and Core Operations

Listing 2: Board Initialization and Operations (oxo.py)

```
1 class Board:
2     def reset_board(self):
3         # A list of lists (2D array)
4         self.pieces = [['.', '.', '.'],
5                         ['.', '.', '.'],
6                         ['.', '.', '.']]
7         self.count = 0 # number of pieces already played
8
9         # is there still somewhere to play (O(1))
10    def unfinished(self):
11        return self.count < 9
12
13    # play a move (O(1))
14    def play(self, pos, player):
```

The Critical Operation: Win Detection I

The `wins(player)` method checks the 8 possible winning lines (3 rows, 3 columns, 2 diagonals).

Listing 3: Board.wins (oxo.py)

```
1  def wins(self, player):
2      colcounts = [0,0,0]
3      d1count = 0 # Diagonal \
4      d2count = 0 # Diagonal /
5      for y in range(0,3):
6          rowcount = 0
7          for x in range(0,3):
8              if self.pieces[y][x] == player:
9                  rowcount+=1
```

The Critical Operation: Win Detection II

```
10         colcounts[x] += 1
11         if colcounts[x] == 3: return True
12         if x == y: # Main diagonal condition
13             d1count += 1
14             if d1count == 3: return True
15         if x == 2 - y: # Anti-diagonal condition
16             d2count += 1
17             if d2count == 3: return True
18         if rowcount == 3: return True
19     return False
```

The Critical Operation: Win Detection III

Algorithm Analysis

This implementation is highly efficient. It iterates through the 9 cells exactly once, simultaneously accumulating counts for all potential winning lines. Complexity is $\mathcal{O}(1)$.

Test-Driven Development (TDD) and `test_board.py`

Before implementing the AI, we must ensure the rules are correct.

The Role of `test_board.py`

Provides unit tests to verify the functional correctness of the Board class in isolation, particularly the crucial `wins()` method.

Listing 4: Excerpt from `test_board.py`

```
1 def test_board():
2     board = Board()
3     # Test Case: Main Diagonal Win
4     board.set("X..",
5              ".X.",
6              "..X")
7     assert(board.wins("X"))
```

Implementation Analysis: The Minimax AI (oxo.py)

Implementation Analysis: The Minimax AI (oxo.py)

Implementation Strategy: Symbolic Outcomes

The `oxo.py` implementation uses symbolic constants rather than numerical utilities (+1, -1, 0).

Listing 5: Outcome Constants (`oxo.py`)

```
1 WIN_IMMEDIATE = 0 # Win on this turn
2 WIN = 1          # Win eventually (forced win)
3 LOSE = 2         # Lose eventually (assuming optimal opponent)
4 DRAW = 3
```

Equivalence to Minimax

This is functionally equivalent. The algorithm prioritizes outcomes according to the following preference relation:

$$\text{WIN_IMMEDIATE} > \text{WIN} > \text{DRAW} > \text{LOSE}$$

Handling State During Search: The Need for Cloning

When exploring hypothetical moves during the recursive search, we must not modify the current game state.

The Problem: Backtracking

If we modify the board in-place, we must "undo" the move before trying the next alternative. This is complex to manage.

The Solution: Cloning (Deep Copy)

Create an independent copy of the game state before simulating a move.

Listing 6: Oxo.clone (oxo.py)

```
1 from copy import deepcopy
2 # ...
3 class Oxo:
```

Handling State During Search: The Need for Cloning

When exploring hypothetical moves during the recursive search, we must not modify the current game state.

The Problem: Backtracking

If we modify the board in-place, we must "undo" the move before trying the next alternative. This is complex to manage.

The Solution: Cloning (Deep Copy)

Create an independent copy of the game state before simulating a move.

Listing 7: Oxo.clone (oxo.py)

```
1 from copy import deepcopy
2 # ...
3 class Oxo:
```

Handling State During Search: The Need for Cloning

When exploring hypothetical moves during the recursive search, we must not modify the current game state.

The Problem: Backtracking

If we modify the board in-place, we must "undo" the move before trying the next alternative. This is complex to manage.

The Solution: Cloning (Deep Copy)

Create an independent copy of the game state before simulating a move.

Listing 8: Oxo.clone (oxo.py)

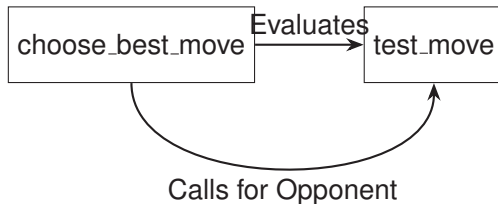
```
1 from copy import deepcopy
2 # ...
3 class Oxo:
```

The Recursive Structure: Mutual Recursion

The Minimax algorithm is implemented using **mutual recursion** between two methods in the `Oxo` class, modeling the alternating turns.

`choose_best_move(player)`: Iterates over possible moves and selects the best one (The MAX role).

`test_move(pos, player)`: Evaluates a specific move by recursively calling `choose_best_move` for the opponent (Simulating the MIN role).



The Recursive Core: test_move (I)

This function evaluates the consequence of a specific move.

Listing 9: Oxo.test_move (oxo.py)

```
def test_move(self, pos, player):  
    # 1. Clone the state and simulate the move  
    game = self.clone()  
    game.board.play(pos, player)  
  
    # 2. Base Case 1: Check for immediate win  
    if game.board.wins(player):  
        return WIN_IMMEDIATE  
  
    # 3. Recursive Step: Evaluate the opponent's best response  
    # This assumes the opponent will play optimally (MIN logic)  
    result = game.choose_best_move(other_player(player))
```

The Recursive Core: test_move (II) - The Adversarial Flip

The crucial insight is how the recursive result is interpreted.

Listing 10: Oxo.test_move - Interpretation (oxo.py)

```
79 # 4. Interpret the result (The Minimax inversion)
80 if result == WIN:
81     # If the opponent's best outcome is a WIN, this move is
      a LOSS for us
82     return LOSE
83 elif result == LOSE:
84     # If the opponent's best outcome is a LOSS, this move
      is a WIN for us
85     return WIN
86 return DRAW
```

The Zero-Sum Logic

The Driver: choose_best_move (I) I

This function iterates over all possible moves and selects the one that yields the best Minimax outcome (The MAX role).

Listing 11: Oxo.choose_best_move (oxo.py)

```
1 def choose_best_move(self, player):
2     # Base Case 2: Board is full (Draw)
3     if self.board.unfinished() == False:
4         return DRAW
5
6     # Initialize variables to track the best move found so far
7     winning_move = None; drawing_move = None; losing_move =
8     None
```


The Driver: choose_best_move (I) II

```
9      # Iterate over all empty squares
10     for y in range(0,3):
11         for x in range(0,3):
12             # ... (Check if empty) ...
13             # Evaluate the move using the recursive helper
14             result = self.test_move(pos, player)
15             # ... (Categorization and Selection) ...
```

The Driver: choose_best_move (II) - Optimization and Selection I

Categorizing the results and selecting the best option.

Listing 12: Oxo.choose_best_move - Categorization (oxo.py)

```
50     # ...
51     result = self.test_move(pos, player)
52
53     # Optimization: Stop immediately if an immediate
54     # win is found
55     if result == WIN_IMMEDIATE:
56         self.board.play(pos, player)
57         return WIN
58
59     # Record the move based on its outcome
```

The Driver: choose_best_move (II) - Optimization and Selection II

```
59         if result == WIN: winning_move = pos
60         if result == DRAW: drawing_move = pos
61         if result == LOSE: losing_move = pos
62
63     # Select the best outcome found (Prioritize Win > Draw >
        Lose)
64     if winning_move:
65         # ... (Play winning move and return WIN) ...
66     if drawing_move:
67         # ... (Play drawing move and return DRAW) ...
68     # ... (Handle losing move) ...
```

The Driver: `choose_best_move` (II) - Optimization and Selection III

The MAX Logic

The final selection implements the MAX preference: prioritize a guaranteed Win, settle for a Draw if necessary, and accept a Loss only if unavoidable.

Testing and Analysis

Testing and Analysis

Testing the Strategy (test_oxo.py)

We use unit tests to verify the AI makes optimal decisions in specific scenarios. This is crucial for complex recursive algorithms.

Listing 13: Excerpt from test_oxo.py

```
1 def test_gameplay():
2     game = Oxo()
3     # Scenario 1: Immediate Win Available
4     game.board.set("XO.",
5                    "XO.",
6                    "...")
7     # X should play at (0,2) to win immediately
8     assert(game.choose_best_move("X") == WIN)
9
10    # Scenario 2: Forcing a Draw (Blocking)
11    game.board.set("XOX",
12                  "OXX",
13                  "...")
```

Testing Edge Cases: Immediate vs Future Wins

A crucial test ensures the AI prioritizes immediate wins (as implemented by the WIN_IMMEDIATE short-circuit).

Listing 14: test_oxo.py (Prioritization)

```
1  # extra test to ensure that 0 wins immediately if it can,
   rather
2  # than winning sometime later.
3  game.board.set("X.X",
4                 "00.",
5                 "X..")
6  # 0 can win immediately at (2,1) [Board coordinates (x,y)]
7  assert(game.choose_best_move("0") == WIN)
8
9  # Crucially, verify that the immediate winning move was taken
10 # (Coordinates x=2, y=1 map to pieces[1][2])
    assert(game.board.pieces[1][2] == "0")
```

Complexity Analysis of Minimax

Properties

Completeness: Yes, if the game tree is finite.

Optimality: Yes, it guarantees the optimal strategy against a rational opponent.

Complexity

Time Complexity: $\mathcal{O}(b^d)$. It performs a complete DFS of the game tree.

Space Complexity: $\mathcal{O}(d)$. Proportional to the depth of the tree (for the recursion stack).

The Limitation

The exponential time complexity $\mathcal{O}(b^d)$ makes Minimax intractable for complex games.

Complexity Analysis of Minimax

Properties

Completeness: Yes, if the game tree is finite.

Optimality: Yes, it guarantees the optimal strategy against a rational opponent.

Complexity

Time Complexity: $\mathcal{O}(b^d)$. It performs a complete DFS of the game tree.

Space Complexity: $\mathcal{O}(d)$. Proportional to the depth of the tree (for the recursion stack).

The Limitation

The exponential time complexity $\mathcal{O}(b^d)$ makes Minimax intractable for complex games.

Complexity Analysis of Minimax

Properties

Completeness: Yes, if the game tree is finite.

Optimality: Yes, it guarantees the optimal strategy against a rational opponent.

Complexity

Time Complexity: $\mathcal{O}(b^d)$. It performs a complete DFS of the game tree.

Space Complexity: $\mathcal{O}(d)$. Proportional to the depth of the tree (for the recursion stack).

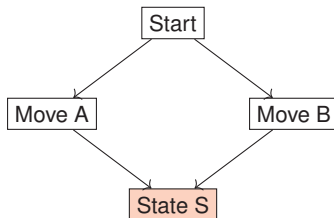
The Limitation

The exponential time complexity $\mathcal{O}(b^d)$ makes Minimax intractable for complex games.

Optimization 1: Memoization (Transposition Tables)

The Problem: Redundancy

The game tree often contains identical states reached through different sequences of moves (transpositions). Naive Minimax re-evaluates these repeatedly.



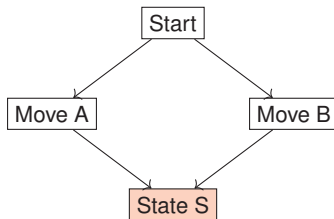
State S is evaluated twice

The Solution: Caching Results

Optimization 1: Memoization (Transposition Tables)

The Problem: Redundancy

The game tree often contains identical states reached through different sequences of moves (transpositions). Naive Minimax re-evaluates these repeatedly.



State S is evaluated twice

The Solution: Caching Results

Optimization 2: Alpha-Beta Pruning

Can we avoid exploring branches that will not influence the final decision? Yes.

The Principle of Pruning

If we have already found a move that guarantees a certain outcome, we do not need to explore other moves that we can already determine will lead to a worse outcome.

Intuitive Example

MAX has found a move A guaranteeing a score of 5. MAX is now evaluating move B. If MAX realizes that MIN has a response to B that leads to a score of 3, MAX can immediately stop evaluating B (prune). Since $3 \nless 5$, move B will not be chosen.

Impact

Alpha-Beta Pruning is the standard optimization. In the best case, it reduces the complexity

Optimization 2: Alpha-Beta Pruning

Can we avoid exploring branches that will not influence the final decision? Yes.

The Principle of Pruning

If we have already found a move that guarantees a certain outcome, we do not need to explore other moves that we can already determine will lead to a worse outcome.

Intuitive Example

MAX has found a move A guaranteeing a score of 5. MAX is now evaluating move B. If MAX realizes that MIN has a response to B that leads to a score of 3, MAX can immediately stop evaluating B (prune). Since $3 \nless 5$, move B will not be chosen.

Impact

Alpha-Beta Pruning is the standard optimization. In the best case, it reduces the complexity

Optimization 2: Alpha-Beta Pruning

Can we avoid exploring branches that will not influence the final decision? Yes.

The Principle of Pruning

If we have already found a move that guarantees a certain outcome, we do not need to explore other moves that we can already determine will lead to a worse outcome.

Intuitive Example

MAX has found a move A guaranteeing a score of 5. MAX is now evaluating move B. If MAX realizes that MIN has a response to B that leads to a score of 3, MAX can immediately stop evaluating B (prune). Since $3 \nless 5$, move B will not be chosen.

Impact

Alpha-Beta Pruning is the standard optimization. In the best case, it reduces the complexity

Summary

Summary

Lecture Summary I

Fundamentals

- **Philosophy:** Understanding core algorithms ("Inside the Lift") is essential for computer scientists.
- **Recursion:** A fundamental technique (Base Case + Recursive Step) essential for traversing tree structures.

Adversarial Search and Game Trees

- Modeling competitive environments using Game Trees.
- Complexity $\mathcal{O}(b^d)$ dictates feasibility.

Lecture Summary II

The Minimax Algorithm

- The foundational algorithm for optimal play in zero-sum games (MAX maximizes, MIN minimizes).
- Implemented using recursive Depth-First Search.

Implementation Details

Lecture Summary III

- The Python implementation used mutual recursion to model alternating turns.
- State management via `deepcopy` is crucial for correct search.
- Rigorous testing (TDD) validates both the rules (`test_board.py`) and the strategy (`test_oxo.py`).
- Optimizations (Alpha-Beta Pruning) are necessary for complex games.